Introduction to Computer Science and Programming

Labo 18 - Disciplined Development with LLMs

Using ChatGPT & GitHub Copilot Responsibly in C++ Development

Laboratory 18, April 16, 2026

Filippo Ricca - Muhammad Mohsin

DIBRIS, University of Genova

Today…

Today's lab is dedicated to **disciplined development using LLMs**

- What are LLMs? (ChatGPT & GitHub Copilot)
- How LLMs generate code — and their **limitations**
- Prompt engineering: **Good** vs. **Bad** prompts
- The **Review → Revise → Test** workflow
- Hands-on Exercises (8 exercises using ChatGPT & Copilot)

All C++ programs must be delivered on Aulaweb!

What are LLMs?

Large Language Models (**LLMs**) are AI systems trained on massive text datasets.

They predict the **next token** (word/symbol) based on context.

They can generate code, explain concepts, and debug errors.

Popular tools for developers:

- **ChatGPT** — Conversational AI by OpenAI (chat interface)
- **GitHub Copilot** — AI pair programmer integrated into VS Code

Important: LLMs don’t “understand” code — they predict patterns!

ChatGPT for C++ Development

ChatGPT is a conversational AI that can help you write, debug, and understand C++ code.

What you can do:

- Ask it to **generate** a function or class from a description
- Ask it to **explain** unfamiliar syntax or error messages
- Paste your code and ask it to **find bugs**
- Ask for **test cases** to verify your solution

Important: Always **review** and **test** every line ChatGPT produces!

GitHub Copilot in VS Code

GitHub Copilot is an AI pair programmer that works **inside your editor**.

How it works:

- Suggests **inline completions** as you type (gray ghost text)
- Press **Tab** to accept, **Esc** to dismiss
- Write a **comment** describing the function → Copilot generates it
- Also available via **Copilot Chat** panel in VS Code

Remember: Copilot is a **suggestion tool**, not an oracle. You must review!

Setting Up Your Environment

Make sure you have these tools ready before starting the exercises:

- **VS Code** with C/C++ extension installed
- **g++ compiler** (MinGW on Windows, gcc on Linux/Mac)
- **GitHub Copilot** extension in VS Code (free with student account)
- **ChatGPT** access via browser (chatgpt.com)

Hint: Apply for the **GitHub Student Developer Pack** for free Copilot access.

The Golden Rules of Using LLMs

1. **Never submit code you don't understand**
2. **Always compile and test** before submitting
3. Write **specific prompts** — vague prompts produce vague code
4. Use the **Review → Revise → Test** cycle
5. Check for **deprecated** or **unsafe** functions (e.g. `gets()`, raw `new`)
6. LLMs **hallucinate** — they confidently produce wrong code

You are the developer. The LLM is just a tool.

Exercise 1: Generate a Function with ChatGPT

Ask ChatGPT to write a C++ function **isPrime(int n)** that returns **`true`** if the number is prime, **`false`** otherwise. Then write a **`main()`** that tests it with values 1–20.

- **Review** the generated code line by line
- **Compile** and run it in VS Code
- Does it handle **edge cases**? (0, 1, negative numbers)

Hint: A good prompt specifies the function signature, return type, and test expectations.

Exercise 2: Prompt Engineering – Good vs. Bad

Ask ChatGPT to write a program that reads an array of integers from the user and finds the **second largest** element.

Try **two prompts** and compare the output:

- **Bad:** `“Write C++ code for second largest”`
- **Good:** `“Write a C++ program that reads n integers into a dynamic array, then finds the second largest using a single pass. Handle duplicates.”`

Compare: Which output is **more correct**, **better structured**, and **handles edge cases**?

Exercise 3: Copilot Autocomplete Challenge

Create a new file `stats.cpp` in VS Code. Write only a **comment** describing the function, then let Copilot generate the body.

```
// Function that takes a vector of doubles
// and returns the mean, min, and max
```

- Accept the suggestion with **Tab**
- **Review** the generated code: does it compile? Is the logic correct?
- Test with: `{3.5, 7.2, 1.1, 9.8, 4.6}`

Exercise 4: Spot the Bug in LLM Code

Ask ChatGPT: “Write a C++ function that reverses a C-string **in place**.”

The generated code **may contain bugs**. Your task:

1. Copy the code into VS Code and **compile** it
2. Test with: “`hello`”, “” (empty), “a” (single char)
3. Identify any **off-by-one** or **null-terminator** errors
4. Fix the bug and re-test

Exercise 5: Dynamic Array with Copilot

Using Copilot in VS Code, write a program that **dynamically allocates** an integer array, lets the user fill it, and prints the **sum** and **average**.

- Start by typing a comment, let Copilot suggest the code
- Check: does Copilot use **new** and `delete[]` correctly?
- Verify there are **no memory leaks**
- Test with: size = 5, values = {10, 20, 30, 40, 50}

Don't forget to deallocate the dynamic memory to avoid memory leaks

Exercise 6: Refactor with ChatGPT

Take your solution from **Exercise 5** and paste it into ChatGPT with the prompt:

```
“Refactor this code to use functions,
add input validation, and add comments.”
```

- Compare the **original** vs. **refactored** version
- Did ChatGPT **change the logic**? Does it still produce the same output?
- Compile, test, and verify the refactored version

Exercise 7: Memory Leak Detection

Ask ChatGPT to generate a C++ program that **intentionally contains a memory leak** (allocates an array, reassigns the pointer without freeing).

1. Read the code and **identify where the leak occurs**
2. Explain **why** the leak happens
3. Fix the code so all memory is properly `delete[]`
4. Compile and verify the fix

Don't forget to deallocate the dynamic memory to avoid memory leaks

Exercise 8: Full Program with Review Cycle

Use ChatGPT **and** Copilot together to build a **student grade manager**:

- Dynamic array of `struct Student { string name; float grade; }`
- Functions: `addStudent`, `printAll`, `averageGrade`, `findBestStudent`
- Proper `new` / `delete[]` memory management

Apply the full **Review → Revise → Test** cycle before submitting.

Common Anti-Patterns with LLMs

What NOT to do:

- **Copy-paste without reading** — you won't learn, and bugs slip in
- **Trusting LLM output blindly** — it can be subtly wrong
- **Using deprecated functions** the LLM suggests (e.g. `gets()`)
- **Skipping compilation** — "it looks right" is not testing
- **Asking the LLM to "fix it" repeatedly** without understanding the error

Treat the LLM as a junior colleague — review everything it produces.

The Review → Revise → Test Workflow

Every time you get code from an LLM, follow this cycle:

1. **Review** — Read every line. Do you understand it?
2. **Revise** — Fix bugs, rename variables, add comments
3. **Test** — Compile, run with normal **and** edge-case inputs

If tests fail → go back to step 1. **Do not** just ask the LLM to fix it without understanding the error yourself.

This workflow is the core discipline of this lab.

Before You Submit: Checklist

- ☐ I can **explain every line** of my code
- ☐ The code **compiles without warnings** (`g++ -Wall -Wextra`)
- ☐ I tested with **normal inputs** and **edge cases**
- ☐ All **dynamic memory** is properly freed (no leaks)
- ☐ I did **not** use deprecated or unsafe functions
- ☐ Variable names are **meaningful** and code is **commented**
- ☐ All exercises uploaded to **Aulaweb**